

SpoiledOrNot: Real-Time Fruit Freshness Detection Using Deep Learning

Vincent Griest

Oakland University

vmgriest@oakland.edu

April 9, 2026

Abstract

Food waste and foodborne illness remain significant global challenges, with spoilage detection often relying on subjective human judgment. This paper presents *SpoiledOrNot*, a deep learning system for automated food freshness classification. I developed a ResNet-18-based classifier capable of distinguishing between 18 fresh and spoiled food categories. The model achieves 98.71% validation accuracy on a held-out validation set (20% of training data). However, critical evaluation on the official test split reveals a significant performance drop to 24.98% accuracy, highlighting a severe class mismatch between training and test distributions. I provide detailed analysis of this discrepancy, identifying missing classes in the test set as the primary cause. The system includes both command-line and web interfaces for real-world deployment. I comprehensively evaluate performance using accuracy, precision, recall, F1-score, confusion matrices, and ROC-AUC analysis (macro AUC = 0.9997 on validation).

Keywords: Computer Vision, Deep Learning, Food Freshness, Convolutional Neural Networks, ResNet-18, Transfer Learning, Class Imbalance

1 Introduction

1.1 Problem Motivation

Approximately one-third of all food produced globally is wasted each year, representing nearly 1.3 billion tons of food loss [Gustavsson et al. \(2011\)](#). A significant portion of this waste occurs due to improper storage, inadequate quality assessment, and subjective spoilage evaluation. Simultaneously, foodborne illnesses affect an estimated 600 million people annually, with contaminated produce being a major contributor [WHO \(2015\)](#).

Traditional methods of assessing food freshness rely heavily on human visual inspection, which is subjective, inconsistent, and requires trained expertise. Consumers often struggle to identify early-stage spoilage, leading to either premature disposal of edible produce or consumption of compromised food. There exists a clear need for automated, objective, and accessible systems capable of real-time food freshness assessment.

1.2 Project Objectives

This project aims to develop an intelligent food freshness detection system with the following objectives:

1. **High Accuracy:** Achieve $>95\%$ classification accuracy on validation data
2. **Generalization:** Maintain performance across held-out test distributions
3. **Real-time Performance:** Enable practical deployment with reasonable inference speed
4. **Accessibility:** Provide both desktop and web interfaces

1.3 Key Findings

Our experiments reveal a critical insight: while the model achieves excellent performance on the validation set (98.71%), test set performance degrades dramatically to 24.98%. Analysis identifies a class mismatch between training (18 classes) and test (14 classes) as the primary cause. This finding underscores the importance of dataset consistency in machine learning pipelines.

2 Related Work

2.1 Traditional Computer Vision Methods

Early approaches to food quality assessment relied on hand-crafted features including color histograms, texture analysis (LBP, GLCM), and shape descriptors [Bhargava & Bansal \(2013\)](#). These methods required domain expertise for feature engineering and struggled with lighting variations and diverse fruit orientations.

2.2 Deep Learning Approaches

Recent work has demonstrated superior performance using CNNs for food quality assessment:

- **Zhang et al. [Zhang et al. \(2020\)](#):** Proposed a multi-task CNN for simultaneous fruit detection and freshness estimation, achieving 94.2% accuracy on a custom dataset.
- **Patel et al. [Patel et al. \(2021\)](#):** Developed a mobile-optimized EfficientNet-based system for fruit classification with deployment on edge devices.
- **Kumar et al. [Kumar et al. \(2019\)](#):** Investigated ensemble methods combining ResNet, DenseNet, and MobileNet for enhanced robustness.

2.3 Our Contribution

This project differentiates itself through:

1. **Comprehensive Error Analysis:** Detailed investigation of train-test class mismatch
2. **Transparent Reporting:** Honest documentation of performance discrepancies
3. **Educational Focus:** Detailed metric explanations suitable for learning contexts
4. **Practical Deployment:** Working CLI and web interfaces

3 Data

3.1 Dataset Source

I utilize the “Fresh and Stale Classification” dataset from Kaggle [Swoyam \(2023\)](#). The dataset contains images across multiple fresh and spoiled food categories. Table 1 summarizes the dataset composition.

Table 1: Dataset Composition by Split

Split	Classes	Total Samples	Images per Class
Training	18	~15,000+	50-650
Validation (20% holdout)	18	~4,000+	10-160
Test (official)	14	6,738	224-988

3.2 Class Distribution Discrepancy

A critical issue emerged during evaluation: the training set contains 18 classes while the test set contains only 14 classes. The missing classes are:

- freshbittergroud
- freshcapsicum
- rottenbittergroud
- rottencapsicum

This mismatch fundamentally breaks the evaluation protocol and explains the dramatic performance drop from validation to test.

3.3 Data Preprocessing

All images undergo standardized preprocessing. Training transforms include:

```
Resize(224x224) -> RandomHorizontalFlip(p=0.5) ->
RandomRotation(15) -> ColorJitter(brightness=0.2,
                                   contrast=0.2) ->
Normalize(mean=[0.485, 0.456, 0.406],
          std=[0.229, 0.224, 0.225])
```

Validation and test transforms exclude augmentation, using only resize and normalization.

4 Methods

4.1 Model Architecture

4.1.1 ResNet-18 Backbone

I employ ResNet-18 [He et al. \(2016\)](#) pre-trained on ImageNet [Deng et al. \(2009\)](#) as the feature extractor. The architecture consists of:

- 5-stage convolutional backbone with residual connections
- 18 layers total (convolutional + fully connected)
- 11.7 million parameters
- Final fully-connected layer modified for 18 output classes

The classification head transforms features from dimension 512 to 18 classes:

$$\text{FC}_{\text{out}} : \mathbb{R}^{512} \rightarrow \mathbb{R}^{18} \quad (1)$$

4.2 Training Configuration

Training hyperparameters are detailed in [Table 2](#).

Table 2: Training Configuration

Parameter	Value
Learning Rate (initial)	1×10^{-4}
Optimizer	Adam
Scheduler	Cosine Annealing
Batch Size	32
Epochs	12
Loss Function	Cross-Entropy
Device	CPU (fallback)

4.3 Evaluation Metrics

I report standard classification metrics:

- **Accuracy:** $\frac{\text{TP} + \text{TN}}{\text{Total}}$
- **Precision:** $\frac{\text{TP}}{\text{TP} + \text{FP}}$
- **Recall:** $\frac{\text{TP}}{\text{TP} + \text{FN}}$

- **F1-Score:** $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$
- **ROC-AUC:** Area under Receiver Operating Characteristic curve

5 Experiments & Results

5.1 Training Dynamics

The model was trained for 12 epochs with cosine annealing learning rate scheduling. Figure 1 shows training loss decreasing from 0.455 to 0.045, while validation accuracy improved from 95.34% to a peak of 98.71% at epoch 10.

Table 3: Training Progress (Selected Epochs)

Epoch	Train Loss	Val Accuracy	Learning Rate
1	0.4548	95.34%	0.000098
3	0.1806	95.62%	0.000085
6	0.1079	97.50%	0.000050
9	0.0624	98.14%	0.000015
10	0.0521	98.71%	0.000007
12	0.0453	98.67%	0.000000

5.2 Validation Performance

On the validation set (20% holdout from training data), the model achieves:

Table 4: Overall Validation Performance

Metric	Value
Accuracy	98.71%
Macro Precision	0.9725
Macro Recall	0.9721
Macro F1-Score	0.9722
ROC-AUC (macro)	0.9997

5.2.1 Per-Class Validation Performance

Table 5 shows per-class metrics on validation data. Most classes achieve near-perfect scores, with lower performance on challenging categories like cucumber (F1=0.9535) and okra (F1=0.9483).

Table 5: Per-Class Validation Performance (Selected Classes)

Class	Precision	Recall	F1-Score
freshapples	1.0000	1.0000	1.0000
freshbanana	0.9959	0.9980	0.9969
freshcucumber	0.9318	0.9762	0.9535
freshokra	0.9322	0.9649	0.9483
rottenapples	1.0000	0.9969	0.9984
rottencucumber	0.9024	0.8605	0.8810
rottenokra	0.8919	0.8571	0.8742

5.3 Test Performance Crisis

When evaluated on the official test split, performance collapses dramatically:

Table 6: Test Performance (Official Split)

Metric	Value
Accuracy	24.98%
Macro Precision	0.1426
Macro Recall	0.1429
Macro F1-Score	0.1427

5.3.1 Analysis of Performance Collapse

The dramatic performance drop (98.71% $\rightarrow$ 24.98%) is explained by three factors:

1. **Class Mismatch:** The test set contains only 14 of 18 training classes. The model outputs probabilities for all 18 classes, but evaluation only considers the 14 present classes, creating systematic scoring errors.
2. **Inconsistent Class Naming:** Test set uses labels like “freshpatato” (misspelled) and “freshtamto” versus training’s “freshpotato” and “freshtomato”.
3. **Distribution Shift:** Even for matching classes, test samples may come from different sources or conditions.

Table 7 shows which classes achieved non-zero performance:

Table 7: Test Performance by Class

Class	Precision	Recall	Samples
freshapples	1.0000	1.0000	791
freshbanana	0.9966	1.0000	892
All other 12 classes	0.0000	0.0000	5,055

Only apples and bananas achieve correct classification; all other 12 classes fail completely.

5.4 Confusion Matrix Analysis

The confusion matrix (Table 8) reveals systematic misclassification patterns. For example, freshcucumber (279 samples) is consistently misclassified as freshoranges (278 samples), indicating the model learned spurious correlations rather than actual freshness features.

Table 8: Confusion Matrix Excerpt (Test Set)

True Label	Predicted Label	Count	Error Rate
freshcucumber	freshoranges	278	99.6%
freshokra	freshpatato	364	98.4%
freshoranges	freshtamto	386	99.5%
freshpatato	rottenapples	251	92.9%
freshtamto	rottenbanana	251	98.4%

6 Discussion

6.1 Why Validation Succeeded But Test Failed

The excellent validation performance (98.71%) suggests the model successfully learned to distinguish the 18 training classes. However, the validation set was created by randomly holding out 20% of training data, meaning it shares the same distribution as training.

The test set, conversely, comes from a different distribution with:

- Different class taxonomy (14 vs 18 classes)
- Inconsistent class naming conventions
- Potentially different image sources or preprocessing

This represents a classic “dataset shift” problem where evaluation on non-IID data reveals overfitting to training distribution artifacts.

6.2 Implications for Real-World Deployment

For practical deployment, this system would fail catastrophically on unseen food categories. The results highlight:

1. **Critical Importance of Test Distribution:** Validation accuracy alone is insufficient for deployment decisions.
2. **Need for Class Alignment:** Training and test sets must share identical class taxonomies.
3. **Value of Error Analysis:** Confusion matrices revealed systematic misclassification patterns.

6.3 Recommendations

To achieve reliable deployment, I recommend:

1. **Reconcile Class Taxonomies:** Ensure training and test sets share identical class labels
2. **Expand Test Coverage:** Include all food categories present in training
3. **Cross-Validation:** Use k-fold cross-validation within the training distribution
4. **Holdout by Source:** Split by data source rather than randomly to better estimate generalization

7 Conclusion

7.1 Summary of Accomplishments

This project successfully developed SpoiledOrNot, achieving:

1. **Strong Validation Performance:** 98.71% accuracy on held-out validation data
2. **Comprehensive Analysis:** Detailed error analysis revealing train-test mismatch
3. **Practical Deployment:** Working CLI and web interfaces
4. **Educational Value:** Clear demonstration of dataset shift consequences

7.2 Key Lessons

The most significant finding is not the high validation accuracy, but the dramatic test set failure. This provides a valuable case study in:

- The importance of representative test sets
- Dangers of overfitting to training distribution artifacts
- Need for class taxonomy consistency across splits
- Value of confusion matrix analysis for debugging

7.3 Limitations

1. **Dataset Mismatch:** Cannot reliably evaluate on official test split
2. **Class Imbalance:** Some classes have very few samples (e.g., freshbittergourd: 53 samples)
3. **Computational Constraints:** Training performed on CPU only
4. **Binary Fresh/Spoiled Only:** No fine-grained ripeness stages

7.4 Future Work

1. Reconcile class taxonomies between train and test splits
2. Collect additional test data for missing classes
3. Implement domain adaptation techniques for distribution shift
4. Deploy TensorFlow Lite version for mobile devices
5. Add multi-class ripeness stage classification

References

- Gustavsson, J., Cederberg, C., Sonesson, U., Van Otterdijk, R., & Meybeck, A. (2011). Global food losses and food waste. *FAO*.
- World Health Organization. (2015). WHO estimates of the global burden of foodborne diseases. *WHO*.
- Deng, J., Dong, W., Socher, R., Li, L. J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. *CVPR*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *CVPR*.
- Swoyam. (2023). Fresh and Stale Classification Dataset. *Kaggle*.
- Zhang, Y., et al. (2020). Multi-task learning for fruit detection and freshness estimation. *IEEE Access*.
- Patel, H., et al. (2021). Mobile-optimized EfficientNet for fruit quality assessment. *arXiv preprint*.
- Kumar, S., et al. (2019). Ensemble deep learning for food quality assessment. *Computers and Electronics in Agriculture*.
- Bhargava, A., & Bansal, A. (2013). Fruits and vegetables quality evaluation using computer vision. *IJCSIT*.